

SingleStore 트랜잭션 멱등성 설계 가이드 (1)

문서번호 : 11-4697777

Document Information

Goal

Background

분산 데이터베이스의 2PC (Two-Phase Commit)

트랜잭션 커밋 응답 유실과 불확실성

1. 단일 노드 환경

2. 분산 데이터베이스 환경

멱등성이란

멱등성 보장 여부 판단

Solution

1. Error 1997 개요

1-1. 2PC failure의 결과 결정 요인

2. 관련 에러 코드별 처리 기준

3. Transaction Tracking Table 운영

3-1) SELECT으로 보장할 수 없는 이유

3-2) 기존 테이블의 일반 컬럼으로 멱등성을 보장할 수 없는 이유

3-3) 멱등성 보장 방식

4. Transaction Tracking Table 운영 예시

4-1) Tracking Table 생성

4-2) 업무 트랜잭션 수행

5. 권장 구성

Reference

History

Document Information

- 최초 작성일 : 2026.07.20
- 최종 수정일 : 2026.08.11
- 이 문서는 아래 버전을 기준으로 작성되었습니다.
 - SinglestoreDB : 9.0.27

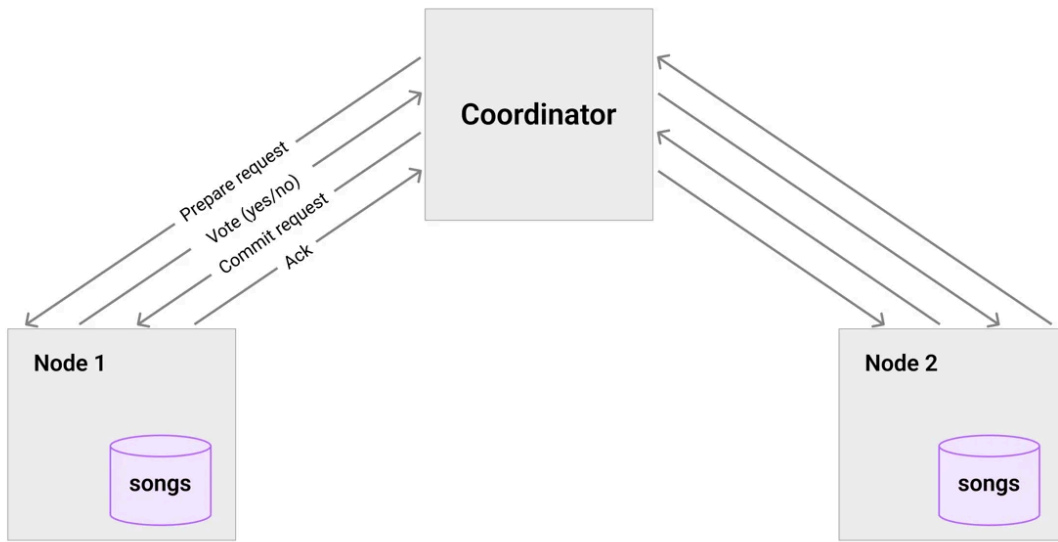
Goal

- 트랜잭션 응답 유실에 따른 불확실성을 설명합니다.
- Error 1997 발생 시, 안전하게 재시도하기 위한 대응 방법(tracking table 기반 멍등성 보장)을 설명합니다.

Background

분산 데이터베이스의 2PC (Two-Phase Commit)

분산 DB 환경에서는 트랜잭션의 변경 사항이 관련된 모든 파티션에 걸쳐 원자성(All-or-Nothing)을 보장하기 위해 코디네이터 노드(Aggregator)의 주도하에 2PC 프로토콜을 수행합니다.



1. Prepare Phase (준비 단계): 코디네이터가 각 Leaf 노드에 커밋 준비를 요청하고, 노드들의 투표(Vote: Yes/No)를 수집합니다.
2. Commit Phase (커밋 단계): 모든 노드가 'Yes'를 응답하면, 코디네이터가 Commit Request를 전송하고 Leaf 노드들로부터 완료 응답(ACK)을 받습니다.

트랜잭션 커밋 응답 유실과 불확실성

트랜잭션의 Commit 처리 결과를 알리는 응답이 클라이언트에게 도달하기 전, 네트워크 장애나 소켓 타임 아웃으로 인해 패킷이 유실되는 현상이 발생할 수 있습니다. 이 경우, 클라이언트는 해당 트랜잭션에 대한 최종 반영 여부 (Commit / Rollback)를 확신할 수 없는 불확실(In-Doubt/Ambiguous) 상태에 빠지게 됩니다.

이 문제는 클라이언트와 서버가 통신하면서 발생하는 구조적 한계이며, 아키텍처(단일 노드 vs 분산 DB)에 따라 발생 레이어가 달라집니다.

1. 단일 노드 환경

단일 서버 아키텍처에서는 장애 발생 지점이 **클라이언트 ↔ 서버** 구간으로 한정됩니다.

트랜잭션의 처리 결과를 클라이언트에게 반환하는 과정에서 커밋 응답 패킷이 유실되거나 타임 아웃이 발생할 수 있습니다. 이 경우, 클라이언트는 해당 트랜잭션에 대한 최종 반영 여부(Commit / Rollback)를 알 수 없는 상태가 됩니다.

MongoDB는 **UnknownTransactionCommitResult** 에러 레이블로 표시하고 있습니다.

2. 분산 데이터베이스 환경

분산 데이터베이스 환경에서는 **클라이언트 ↔ 서버** 구간 외에 **코디네이터 노드 ↔ Leaf 노드** 구간이 추가됩니다.

트랜잭션에 연관된 파티션이 2PC 수행 중 장애로 응답(vote 또는 ack)을 보내지 못하면, 코디네이터는 해당 트랜잭션의 최종 반영 여부(Commit/Rollback)를 확인할 수 없습니다. CockroachDB는 **ambiguous error** 에러 레이블로 표시하고 있습니다.

멱등성이란

멱등성(idempotency)은 동일한 연산을 여러 번 수행해도 최종 결과가 한 번 수행한 것과 동일하게 유지되는 성질을 의미합니다.

멱등한 연산

- `UPDATE users SET status = 'inactive' WHERE user_id = 1;`
몇 번을 실행해도 결과는 항상 status가 'inactive'인 상태로 동일합니다.
- `DELETE FROM sessions WHERE session_id = 'abc';`
이미 삭제된 row를 다시 삭제 시도해도, 최종적으로 해당 row가 없다는 결과는 동일합니다.

멱등하지 않은 연산

- `UPDATE inventory SET count = count - 1 WHERE item_id = 1;`
실행할 때마다 count 값이 계속 감소하므로, 여러 번 실행하면 결과가 달라집니다.
- `INSERT INTO orders (amount) VALUES (100);`
실행할 때마다 새로운 row가 생성되므로, 여러 번 실행하면 중복 데이터가 쌓입니다.

멱등성 보장 여부 판단

멱등성 보장의 여부는 비즈니스 로직 관점에서 판단해야 하는 문제입니다.

트랜잭션이 재시도 가능성이 있고 그 결과가 중복 반영되었을 때, 문제(이중 결제, 재고 오차 등)가 발생한다면 멱등성 보장을 고려해야 합니다. 그러나 결과가 여러 번 반영되어도 비즈니스적으로 문제가 되지 않는 트랜잭션이라면 멱등성을 보장하지 않아도 됩니다.

Solution

1. Error 1997 개요

SingleStore에서도 위와 같은 현상이 발생하며, 장애가 발생한 파티션이 다시 Online 상태가 되어 2PC resolution protocol이 완료될 때까지 트랜잭션의 최종 결과(Commit/Rollback)를 확인할 수 없는 상태가 지속됩니다.

```
OperationalError: 1997: Attempted to interrupt transaction execution,
but the outcome is unknown because the transaction was already in its commit phase.
Please check if it succeeded.
```

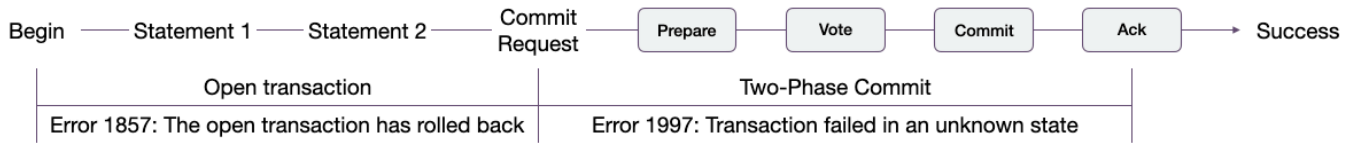
코디네이터는 이처럼 트랜잭션의 최종 결과를 판단할 수 없는 경우 Error 1997을 반환합니다.

1-1. 2PC failure의 결과 결정 요인

Error 1997 발생 이후, 트랜잭션과 연관된 모든 파티션이 장애 발생 시점에 prepare 결정(Yes)을 disk에 기록한 상태였다면 해당 트랜잭션은 Commit으로, 그렇지 않다면 Rollback으로 확정됩니다. 그러므로 트랜잭션의 성공률과 서비스 연속성이 모두 중요한 금융 또는 트랜잭션 시스템에서는 **Sync replication (H/A)**과 **Sync durability**를 함께 구성하는 것이 권장됩니다.

2. 관련 에러 코드별 처리 기준

클러스터 장애 시점별 발생 에러 코드



에러 코드	내용	처리 방식
1997	2PC 수행 중 장애로 transaction의 최종 반영 여부가 확정되지 않음	tracking table과 unique 제약을 통해 멱등성 보장
1857	Open transaction이 Rollback으로 확정됨	즉시 재시도해도 안전하나, 클러스터 장애를 고려해 exponential backoff으로 재시도
1777	해당 파티션의 master instance 부재	node crash recovery가 끝날 때까지 exponential backoff로 재시도

3. Transaction Tracking Table 운영

💡 모든 트랜잭션에 Tracking Table 구축이 필요한 것은 아닙니다.

Tracking Table은 멱등성 보장이 요구되는 트랜잭션에 한해 권장되며, 멱등성 필요 여부 판단 기준은 [Background의 "멱등성 보장 여부 판단"](#) 섹션을 참고하십시오.

트랜잭션의 반영 여부를 알 수 없는 상황에서 안전하게 재시도하려면, 업무 트랜잭션에 멱등성(idempotency)이 요구됩니다. 따라서 멱등성이 요구되는 비즈니스 로직이 포함된 경우, 트랜잭션 고유 식별자(`transaction_id`)를 생성하고 업무 데이터와 함께 별도의 Tracking Table(log성 테이블)에 기록하는 방식을 권장합니다.

3-1) SELECT으로 보장할 수 없는 이유

Error 1997 발생 이후, 원본 transaction의 최종 결과(commit 또는 rollback)는 2PC resolution 이 완료되어야 확정됩니다.

Master Partition failure -> Slave가 Master로 승격 -> 2PC resolution protocol -> Rollback/Commit

이 지연 구간에서 `SELECT ... WHERE transaction_id = %s` 로 조회했을 때 결과가 없다면, 다음 두 가지 경우를 구분할 수 없습니다.

- 2PC resolution 단계에서 Rollback 처리된 경우
- 2PC resolution 아직 완료되지 않아, 이후 commit 될 transaction이 아직 반영되기 전인 경우

따라서 `SELECT` 조회 결과만으로는 원본 transaction의 반영 여부를 확인할 수 없으며, `INSERT` 기반 UNIQUE 제약을 통해 멱등성을 보장해야 합니다.

3-2) 기존 테이블의 일반 컬럼으로 멱등성을 보장할 수 없는 이유

기존 업무 테이블의 PRIMARY KEY는 비즈니스 식별자로 이미 고정되어 있으므로, `transaction_id` 를 PRIMARY KEY로 변경하기 어렵습니다. 또한, Columnstore 테이블은 PRIMARY KEY와 UNIQUE KEY를 동시에 설정할 수 없습니다. 따라서 기존 PRIMARY KEY를 유지하면서 `transaction_id` 에 별도의 UNIQUE KEY를 설정할 수 없습니다.

또한, `transaction_id` 를 단순한 일반 컬럼으로 추가하면 동일한 값의 중복 입력을 데이터베이스가 차단하지 못하므로 멱등성을 보장할 수 없습니다.

따라서, `transaction_id` 에 PRIMARY KEY 또는 UNIQUE KEY를 설정할 수 있는 별도의 Tracking 테이블을 사용해야 합니다.

3-3) 멱등성 보장 방식

`transaction_id` 를 PRIMARY KEY로 설정하면, INSERT 시점에 유일성 검사와 삽입이 하나의 원자적 연산으로 처리됩니다. 따라서 재시도 시점이 원본 트랜잭션의 2PC resolution 완료 전/후 관계없이, 동일한 `transaction_id` 로 INSERT를 시도했을 때 원본과 재시도 트랜잭션 중 하나만 성공하고 나머지는 Duplicate entry 에러(Error 1062)로 실패하여 business logic 의 반복 실행을 방지할 수 있습니다.

애플리케이션은 이 Error 1062를 "원본 트랜잭션이 이미 반영됨"으로 해석하여, 재시도 트랜잭션을 롤백하면 됩니다.

4. Transaction Tracking Table 운영 예시

4-1) Tracking Table 생성

💡 `transaction_id` 는 UUIDv7 사용을 권장합니다. 본 스키마는 `transaction_id` 를 SORT KEY로도 사용하므로, 완전 랜덤인 UUIDv4보다 시간 순서 정렬성을 갖는 UUIDv7이 삽입 시 정렬 유지 비용 측면에서 유리합니다.

```
CREATE TABLE tx_tracking (
  transaction_id BINARY(16) NOT NULL,
  created_at DATETIME(6) DEFAULT NOW(6),
  PRIMARY KEY (transaction_id),
  SHARD KEY (transaction_id),
  SORT KEY (transaction_id)
);
```

`transaction_id` 는 애플리케이션에서 생성하며, PRIMARY/UNIQUE KEY 지정을 통해 멱등성을 보장할 수 있습니다.

4-2) 업무 트랜잭션 수행

- 업무 데이터와 tracking table 기록은 반드시 동일 트랜잭션 내에서 수행해야 합니다.
 - `transaction_id`는 원본 트랜잭션과 동일한 값을 사용해야 합니다.
 - Duplicate Entry (Error 1062)는 원본 transaction이 반영된 것으로 간주합니다.
- 재시도 부하를 줄이기 위해 exponential backoff 사용을 권장합니다.

```
import pymysql
import uuid
import time
```

```

def register_tx(cur, tx_id):
    try:
        cur.execute(
            "INSERT INTO tx_tracking (transaction_id) VALUES (%s)",
            (tx_id,)
        )
        return True

    except pymysql.err.IntegrityError as e:
        if e.args[0] == 1062:
            return False
        raise

def handle_error(e):
    err_code = e.args[0] if e.args else None

    return err_code in (1997, 1857, 1777)

def execute_business_transaction(conn, request):

    tx_id = uuid.uuid7().bytes

    while True:
        cur = conn.cursor()

        try:
            cur.execute("BEGIN")

            if not register_tx(cur, tx_id):
                cur.execute("ROLLBACK")
                return True

            # Existing Business Logic
            cur.execute(
                "INSERT INTO order_data (amount) VALUES (%s)",
                (request["amount"],)
            )

            cur.execute(
                """
                UPDATE account
                SET balance = balance - %s
                WHERE id = %s
                """,
                (
                    request["amount"],
                    request["account_id"]
                )
            )

```

```

    )

    cur.execute("COMMIT")
    return True

except Exception as e:

    if handle_error(e):
        time.sleep(1)
        continue

    try:
        cur.execute("ROLLBACK")
    except Exception:
        pass

    raise

finally:
    cur.close()

```

5. 권장 구성

- **2PC를 활성화 상태로 유지합니다.** v8.5 이상에서 기본값으로 활성화되어 있습니다. 2PC는 하나의 트랜잭션이 여러 파티션에 걸쳐 있을 때 all-or-nothing으로 처리되는 것을 보장합니다.
- **Transaction 식별자 기반의 멍등성 보장 로직을 사용합니다.** 2장에서 설명한 바와 같이, `transaction_id`를 tracking table의 PRIMARY KEY 로 사용하여 재시도 시 중복 반영을 방지합니다.

Reference

- <https://docs.singlestore.com/db/v9.0/query-data/advanced-query-topics/two-phase-transaction-commit/>
- <https://www.mongodb.com/ko-kr/docs/manual/core/transactions-in-applications/>
- <https://www.cockroachlabs.com/docs/v26.2/common-errors#result-is-ambiguous>



Common Errors and Solutions - CockroachDB

<https://www.cockroachlabs.com/docs/v26.2/common-errors#result-is-ambiguous>

Cockroach Labs

Troubleshooting

Common Errors and Solutions

History

<https://www.google.com/>

<https://www.google.com/>



Google

<https://www.google.com/>

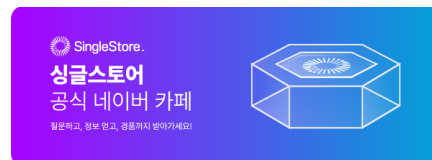
일자	작성자	비고
2026.07.20	mh.Choi	최초 작성
2026.07.22	mh.Choi	내용 수정
2026.07.23	mh.Choi	코드 수정 (retry, backoff)
2026.07.24	mh.Choi	reference 수정 및 1-3 추가
2026.07.31	mh.Choi	4 번 내용 수정
2026.08.11	mh.Choi	Tracking Table 도입 판단 여부 섹션 추가
2026.08.11	wee	문서번호 및 샘플 코드 변경

[illegible]

A-platform



Copyright & Disclaimer



 <https://cafe.naver.com/aplatformbiz>

일자	작성자	비고	비고	비고	비고	비고
2026.07.20	mh.Choi	최초 작성	최초 작성	최초 작성	최초 작성	최초 작성
2026.07.22	mh.Choi	내용 수정	내용 수정	내용 수정	내용 수정	내용 수정
2026.07.23	mh.Choi	코드 수정 (retry, backoff)	코드 수정 (retry, backoff)	코드 수정 (retry, backoff)	코드 수정 (retry, backoff)	코드 수정 (retry, backoff)
2026.07.24	mh.Choi	reference 수정 및 1-3 추가	reference 수정 및 1-3 추가	reference 수정 및 1-3 추가	reference 수정 및 1-3 추가	reference 수정 및 1-3 추가
2026.07.31	mh.Choi	4 번 내용 수정	4 번 내용 수정	4 번 내용 수정	4 번 내용 수정	4 번 내용 수정
2026.08.11	mh.Choi	Tracking Table 도입 판단 여 부 섹션 추가	Tracking Table 도입 판단 여 부 섹션 추가	Tracking Table 도입 판단 여 부 섹션 추가	Tracking Table 도입 판단 여 부 섹션 추가	Tracking Table 도입 판단 여 부 섹션 추가
2026.08.11	wee	문서번호 및 샘플 코드 변 경	문서번호 및 샘플 코드 변 경	문서번호 및 샘플 코드 변 경	문서번호 및 샘플 코드 변 경	문서번호 및 샘플 코드 변 경

일자	작성자	비고	
2026.07.20	mh.Choi	최초 작성	
2026.07.22	mh.Choi	내용 수정	
2026.08.11	wee	문서번호 및 샘플 코드 변경	

```

$ sdb-admin list-nodes
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+
| MemSQL ID | Role | Host | Port | Process State | Connectable? | Version | Recovery
State | Availability Group | Bind Address |
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+
| CA4A5F8A37 | Master | ma | 3306 | Running | True | 7.8.2 | Online
| | | 0.0.0.0 |
| A557CFC555 | Aggregator | ca | 3306 | Running | True | 8.5.16 | Online
| | | 0.0.0.0 |
| 7660E2E74C | Leaf | lf1 | 3306 | Running | True | 8.5.16 | Online
| 1 | | 0.0.0.0 |
| 0EC9E69070 | Unknown | lf2 | 3306 | Running | True | 8.5.16 |
Recovering | | 0.0.0.0 |
+-----+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+

```

```
$ echo hiasd
$ cat
```

 <https://www.google.com/>

 <https://www.google.com/>

 **Google**

<https://www.google.com/>

 **DeepL 번역기 | 세계에서 가장 정확한 AI 번역기**

텍스트 및 전체 문서 파일을 즉시 번역하세요. 개인과 팀을 위한 정확한 AI 번역. 매일 수백만 명이 DeepL 번역기로 작업합니다.

<https://www.deepl.com/ko/translator>

 **DeepL**

 <https://www.google.com/>

▶ 토글 기능은 어떻게 처리되려나?